National University of Singapore

School of Computing
Final Assessment for
Semester 1 AY25/26

CS2030S Programming Methodology II

November 2025 Time Allowed 120 minutes

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains **16 questions** and comprises **16 printed pages**, including this page. The last **3 page(s) are left empty** for your working.
2. Write all your answers in the provided space in provided the answer sheet. Answers not in the provided space will not be graded.
3. Erase any unwanted answers completely. Anything not erased completely will be treated as part of the answer. If a dashed line is provided, each dashed line can only be used for one line.
4. Write legibly. Marks may be deducted for illegible writing.
5. The total marks for this assessment is **60 points**. The mark for each question or sub-question is given in the question.
6. This is a **CLOSED-BOOK** assessment. You are only allowed to refer to **one double-sided A4 cheatsheet**. No electronic devices are allowed including but not limited to calculator, smart-watches, etc.
7. All programs in this assessment paper use Java 21 and are compiled with the flags `-Xlint:unchecked` and `-Xlint:rawtypes`.

Questions	1-12	13	14	15	16	**Total**
Points	24	9	10	10	7	**60**

GOOD LUCK!

Multiple Choice Questions (24 points)

For each question below, **shade your answer in the corresponding bubble on the answer sheet.** If multiple answers are equally appropriate, pick one and shade the chosen answer on the answer sheet.

Do **NOT** shade more than one answer. If none of the answers is appropriate, shade X in the answer sheet.

1. (2 points) Consider the following processes.

   (i). Dynamic binding. (ii). Type inference. (iii). Type erasure.

   Which of the processes above happens at **compile-time** (but not necessarily only at compile-time)?

   A. (i) only
   B. (iii) only
   C. (i) and (ii) only
   D. (i) and (iii) only
   E. (i), (ii) and (iii)

   Shade X in the answer sheet if none of the options are correct.

2. (2 points) Consider the following processes.

   (i). Dynamic binding. (ii). Type inference. (iii). Type erasure.

   Which of the processes above happens at **runtime** (but not necessarily only at runtime)?

   A. (i) only
   B. (iii) only
   C. (i) and (ii) only
   D. (i) and (iii) only
   E. (i), (ii) and (iii)

   Shade X in the answer sheet if none of the options are correct.

For Questions 3 to 4, consider the following code snippet. Assume `V` <: `U` <: `T` <: `S`. Assume the code snippet compiles.

```
T t = new U();
Object obj = (S) t;
```

3. (2 points) What is the compile-time type of `obj`?

   A. `Object`
   B. `S`
   C. `T`
   D. `U`
   E. `V`

   Shade X in the answer sheet if none of the options are correct.

4. (2 points) What is the runtime type of `obj`?

   A. `Object`
   B. `S`
   C. `T`
   D. `U`
   E. `V`

   Shade X in the answer sheet if none of the options are correct.

Consider if this is allowed. `X<-T>` is contravariant but `Y<+T>` is covariant. Consider `A <: B`. This means `Y<A> <: Y<B> <: X<B>`. Now, the code similar to part (b) is no longer safe.

```
X<B> x = new Y<A>(new A()); // assume A <: B
x.set(() -> new B());
```

We cannot assign a value of type `B` into a field of type `A` because `A <: B` (i.e., it implies that `B </: A`).

For Questions 5 to 7, consider the following class.

```
class ParentBox<T> {
  public <U> ParentBox<? extends U> foo(T arg) {
    return null;
  }
}
```

For each question, check if the subclass of `ParentBox` can be compiled with/without warning. Choose from one of the following options.

A. The class compiles without warning.

B. The class compiles with a rawtype warning.

C. The class compiles with an unchecked warning.

D. The class compiles with other warnings.

E. The class does not compile.

Shade X in the answer sheet if none of the options are correct.

5. (2 points)

```
class ChildBox<T> extends ParentBox<T> {
  @Override
  public <R> ParentBox<R> foo(T arg) {
    return null;
  }
}
```

6. (2 points)

```
class ChildBox<T> extends ParentBox<T> {
  public <T> ParentBox<? extends T> foo(T arg) {
    return null;
  }
}
```

7. (2 points)

```
class ChildBox extends ParentBox<String> {
  public <T> ParentBox<? extends T> foo(String arg) {
    return null;
  }
}
```

For Questions 8 to 10, consider the following classes.

```
interface Incinerator<T extends Garbage> {
  void incinerate(T trash);
}

abstract class Garbage {
  public void dispose() {
    System.out.println("Garbage::dispose");
  }
}
class Flammable extends Garbage {
  public void dispose() {
    System.out.println("Flammable::dispose");
  }
}
class Plastic extends Flammable {
  public void dispose() {
    System.out.println("Plastic::dispose");
  }
}
class Inflamable extends Garbage {
  public void dispose() {
    System.out.println("Inflamable::dispose");
  }
}

class Disposal<T extends Garbage> {
  Incinerator<? super T> incinerator;

  public Disposal(Incinerator<? super T> incinerator) {
    this.incinerator = incinerator;
  }

  public void dispose(T item) {
    incinerator.incinerate(item);
  }
}
```

Further consider the following code snippet.

```
Plastic toy = new Plastic();
Flammable flam = new Flammable();
Disposal<Flammable> bin = new Disposal<>(item -> flam.dispose());
bin.dispose(toy);
```

8. (2 points) Which of the following is printed at the end of the execution of the code snippet above?

   A. `Garbage::dispose`
   B. `Flammable::dispose`
   C. `Plastic::dispose`
   D. `Inflamable::dispose`
   E. *Nothing is printed*

   Shade X in the answer sheet if none of the options are correct. Also shade X in the answer sheet if the code snippet does not compile.

9. (2 points) Which of the variables in the code snippet above are effectively final?

   A. `toy` only
   B. `flam` only
   C. `item` only
   D. `toy` and `flam` only
   E. `flam` and `item` only

   Shade X in the answer sheet if none of the options are correct. Also shade X in the answer sheet if the code snippet does not compile.

10. (2 points) Consider the following declarations of `bin`.

    (i). `Disposal<Flammable> bin = new Disposal<>(Garbage::dispose);`

    (ii). `Disposal<Flammable> bin = new Disposal<>(Plastic::dispose);`

    (iii).
    ```
    Flammable flam2 = new Flammable();
    Disposal<Flammable> bin = new Disposal<>(flam2::dispose);
    ```

    Which of the declarations above can compile without error?

    A. (i) only
    B. (iii) only
    C. (i) and (ii) only
    D. (i) and (iii) only
    E. (i), (ii) and (iii)

    Shade X in the answer sheet if none of the options are correct.

For Questions 11 to 12, consider the following code snippet.

```
static void bar() { .. }

static CompletableFuture<Void> foo(int i) {
  return CompletableFuture.runAsync(() -> {
        bar();
        System.out.print(i);
      });
}
```

The method `bar` runs for an indeterminate amount of time. Each call to `bar` may run for a different indeterminate amount of time.

You are advised to check the `CompletableFuture` API on Page 13 to understand the methods used (e.g., `runAsync`, `thenRunAsync`, etc).

11. (2 points) Consider the following code snippet of the main method.

```
public static void main(String[] args) {
  CompletableFuture.anyOf(
        foo(1).thenRunAsync(() -> foo(2)),
        foo(3)
      ).thenRunAsync(() -> foo(4));
  bar();
}
```

Consider the following potential print output (**note:** `System.out.print` does not print a newline).

(i). `34` (ii). `132` (iii). `124`

Recap that `main` may finish when there are still other threads running. In which case, the other threads are automatically stopped before completing. Which of the print outputs above is possible?

A. (i) only
B. (iii) only
C. (i) and (ii) only
D. (i) and (iii) only
E. (i), (ii) and (iii)

Shade X in the answer sheet if none of the options are correct.

12. (2 points) Consider the following code snippet of the main method.

```
public static void main(String[] args) {
  foo(1);
  CompletableFuture.anyOf(foo(2), foo(3))
      .thenRun(() -> foo(4).join())
      .join();
  bar();
}
```

Consider the following potential print output (**note:** `System.out.print` does not print a newline).

(i). `34` (ii). `132` (iii). `124`

Recap that the `main` method may finish when there are still other threads running. In which case, the other threads are automatically stopped before completing. Which of the print outputs above is possible?

A. (i) only
B. (iii) only
C. (i) and (ii) only
D. (i) and (iii) only
E. (i), (ii) and (iii)

Shade X in the answer sheet if none of the options are correct.

Short Answer Questions (36 points)

For each question below, **write your answer (or solution) in the given space on the answer sheet.** Write only within the space provided. Make sure that you write legibly; otherwise, marks may be deducted.

13. (9 points) Consider the following class and interfaces.

```
interface Fn2<T> {
  T f(T t1, T t2);
}
interface Fn3<T> {
  T f(T t1, T t2, T t3);
}
class C {
  public C f1(C c) {
    return null;
  }
  public C f1(C c1, C c2) {
    return null;
  }
}
```

(a) (3 points) For each of the following interfaces, determine if it is a functional interface or not. Answer **YES** if it is a functional interface. Otherwise, if it is not a functional interface or the interface cannot compile, answer **NO**.

```
interface FunA extends Fn2<Integer> { }
interface FunB<T> extends Fn2<T> {
  T f(T t);
}
interface FunC<T> extends Fn2<T>, Fn3<T> {
  T f(T t);
}
```

(b) (3 points) Consider the variable `C obj = new C();`. For each of the assignments, determine which `C::f1` is used. Answer `C::f1(C)`, `C::f1(C, C)`, or **NONE**. **NONE** should be selected only if there is no suitable candidate. Consider each assignment separately.

(i) `Fn2<C> fn = C::f1;`

(ii) `Fn2<C> fn = obj::f1;`

(iii) `Fn3<C> fn = obj::f1;`

(c) (3 points) Consider the following assignment.

```
Fn3<C> fn = C::f1;
```

Translate the assignment statement above into an **anonymous class**.

14. (10 points) Consider the following attempt at `Monad`. We treat two `Monad` as equivalent if they are equivalent on both `val` and `ctx`.

```
class Monad<T> {
  private T val;
  private boolean ctx;

  public Monad(T val, boolean ctx) {
    this.val = val;
    this.ctx = ctx;
  }

  public static <T> Monad<T> of(T val) {
    return new Monad<T>(val, __3__);
  }

  public <R> Monad<R> map(Transformer<T, R> fn) {
    return Monad.fmap(fn).transform(this);
  }

  public static <T, R> __1__ fmap(Transformer<T, R> fn) {
    return __2__;
  }

  public <R> Monad<R> flatMap(Transformer<T, Monad<R>> fn) {
    Monad<R> tmp = this.map(fn).val; // note: this.map(fn.val) cannot compile
    return new Monad<R>(tmp.val, __4__);
  }
}
```

Recap the functor laws.

- Identity Law

  $\forall$ `fct`: `fct.map(x -> x)` $\equiv$ `fct`

- Composition Law

  $\forall$ `fct`, `f`, `g`: `fct.map(x -> f(x)).map(y -> g(y))` $\equiv$ `fct.map(x -> g(f(x)))`

Recap the monad laws.

- Left Identity Law

  $\forall$ `x`, `f`: `Monad.of(x).flatMap(y -> f(y))` $\equiv$ `f(x)`

- Right Identity Law

  $\forall$ `m`: `m.flatMap(x -> Monad.of(x))` $\equiv$ `m`

- Associative Law

  $\forall$ `m`, `f`, `g`: `m.flatMap(x -> f(x)).flatMap(y -> g(y))`
  $\equiv$ `m.flatMap(x -> f(x).flatMap(y -> g(y)))`

(a) (4 points) Implement `Monad::fmap` such that `Monad::map` satisfies the functor laws. Fill in `__1__` with a type and `__2__` with a single expression.

(b) (6 points) For each of the following implementation of `__3__` and `__4__`, determine if they **satisfy** each of the monad laws. On the answer sheet, answer **YES** if they satisfy the corresponding law; `NO` otherwise.

(i) implementation #1

`__3__: true`

`__4__: ((!tmp.ctx) || this.ctx)` (**Note:** `||` is a logical `or` operation)

(ii) implementation #2

`__3__: false`

`__4__: (tmp.ctx != this.ctx)`

15. (10 points) Consider our simplified version of `InfiniteList<T>` called `LazyList<T>`. This `LazyList<T>` uses `Producer<T>` instead of `Lazy<Maybe<T>>`. We do not need `Maybe<T>` as we have no filter. We do not need `Lazy<T>` as we do not memoize.

Consider the following usage of `LazyList<T>` when answering the questions below.

```
class LazyList<T> {
  private Producer<T> head;
  private Producer<LazyList<T>> tail;

  private LazyList(Producer<T> head, Producer<LazyList<T>> tail) {
    this.head = head;
    this.tail = tail;
  }

  public static <T> LazyList<T> iterate(T seed,
      Transformer<T, T> next) {
    return new LazyList<>(() -> seed,
        () -> iterate(next.transform(seed), next));
  }

  public <R> LazyList<R> map(Transformer<T, R> fn) {
    return new LazyList<>(
        () -> fn.transform(this.head.produce()),
        () -> this.tail.produce().map(fn));
  }

  public T get(int n) {
    if (n == 0) {
      return this.head.produce();
    } else {
      return this.tail.produce().get(n - 1);
```

continued on next page ...

... continued from previous page

```
    }
  }
}
```

Given a `LazyList<T>` containing `[v1, v2, v3, v4]`, recap that the `reduce` operation with two parameters (i.e., `reduce(T e, Combiner<T, T, T> f)`) should compute the following value. Note that we use the shorthand `f(x, y)` to mean `f.combine(x, y)`.

```
f(f(f(f(e, v1), v2), v3), v4)
```

Since our `LazyList<T>` has no `Sentinel`, we limit the number of elements using another parameter in the `reduce` method itself. As such, we can have an infinitely long `LazyList`, but we can invoke `reduce(id, fn, 4)` on a `LazyList` instance to reduce the first 4 elements. For example,

```
LazyList.iterate(0, x -> x + 1)
        .map(x -> "" + x)   // convert to String
        .reduce("", (x, y) -> y + x, 4);
```

(a) (2 points) As your first implementation, you claim that it is easy because we have `LazyList::get` already. So, you implemented the following.

```
public T reduce(T id, Combiner<T, T, T> fn, int n) {
  T res = id;
  for (int i = 0; i < n; i += 1) {
    res = fn.combine(res, this.get(i));
  }
  return res;
}
```

How many times does the lambda `x -> x + 1` get invoked using this implementation?

(b) (2 points) You noticed that this invokes the lambda `x -> x + 1` too many times. You then write the following improved code.

```
public T reduce(T id, Combiner<T, T, T> fn, int n) {
  T acc = id;
  LazyList<T> curr = this;
  while (n > 0) {
    acc = fn.combine(acc, curr.head.produce());
    curr = curr.tail.produce();
    n = n - 1;
  }
  return acc;
}
```

How many times does the lambda `x -> x + 1` get invoked using this implementation?

(c) (3 points) **Nair B** claims that you can improve this further. Currently, you are invoking `curr.tail.produce()` at the end, even when it is not going to be used in the next iteration. The improvement is to wrap `curr` in a `Producer` so `tail.produce()` is not invoked until it is really needed. The template is given below.

```
public T reduce(T id, Combiner<T, T, T> fn, int n) {
  T acc = id;
  Producer<LazyList<T>> curr = () -> this;
  while (n > 0) {
    ___1___;
    ___2___;
    ___3___;
    n = n - 1;
  }
  return acc;
}
```

Fill in the 3 blanks above with assignment statements such that the number of times `x -> x + 1` is invoked is smaller than the answer for Part (b). You need not use all lines.

(d) (3 points) Show that the call `.reduce("", (x, y) -> y + x, 4)` is parallelizable by showing that `fn` is associative. Fill in the steps below following the steps shown in the lecture.

```
   fn(fn(x, y), z)
≡  ________1________
≡  ________2________
≡  ________3________
≡  fn(x, fn(y, z))
```

16. (7 points) As we know, a Java array is covariant, but a Java generic is invariant. Although Java arrays being covariant may cause some problems, we may want to define the variance of generics in some cases. You propose the following changes to the Java language specification.

- Generic class `class C<T>` is invariant. This is the same as in the original Java.
- Generic class `class C<+T>` is covariant. I.e., if `X` $<:$ `Y`, then `C<X>` $<:$ `C<Y>`.
- Generic class `class C<-T>` is contravariant. I.e., if `X` $<:$ `Y`, then `C<Y>` $<:$ `C<X>`.

To avoid issues with type safety, as that of the Java array, programmers should only be allowed to use `+T` and `-T` when it is safe to use (i.e., only when the classes allow subtyping without violating type safety). To check that you understand your own proposal, the Java language specification committee decided to test you.

(a) (2 points) Consider the following class. Fill in the blank with either `+T` or `-T` to make the class safe to use. If neither `+T` nor `-T` is safe, answer with `T`.

```
class C<___> {
  private T val;

  public C(T val) {
    this.val = val;
  }

  public <R> R get(Transformer<T, R> fn) {
    return fn.transform(this.val);
  }
}
```

(b) (2 points) Consider the following class. Fill in the blank with either `+T` or `-T` to make the class safe to use. If neither `+T` nor `-T` is safe, answer with `T`.

```
class C<___> {
  private T val;

  public C(T val) {
    this.val = val;
  }

  public void set(Producer<T> prod) {
    this.val = prod.produce();
  }
}
```

(c) (3 points) For each of the following pairs of classes, determine if they are safe to use. On the answer sheet, answer **YES** if the pair is safe, answer **NO** otherwise.

(i). Note that the `T` in `extends X<T>` is not `+T` as it is already declared covariant in `Y<+T>`.

```
class X<+T> { }
class Y<+T> extends X<T> { }
```

(ii). Note that the `T` in `extends X<T>` is not `-T` as it is already declared covariant in `Y<+T>`.

```
class X<-T> { }
class Y<+T> extends X<T> { }
```

(iii).

```
class X<T> { }
class Y<-T> extends X<T> { }
```

CompletableFuture API (Abridged)

runAsync

```
public static CompletableFuture<Void> runAsync(Runnable runnable)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the ForkJoinPool.commonPool() after it runs the given action.

join

```
public T join()
```

Returns the result value when complete, or throws an (unchecked) exception if completed exceptionally. To better conform with the use of common functional forms, if a computation involved in the completion of this CompletableFuture threw an exception, this method throws an (unchecked) CompletionException with the underlying exception as its cause.

thenRun

```
public CompletableFuture<Void> thenRun(Runnable action)
```

Returns a new CompletionStage that, when this stage completes normally, executes the given action.

thenRunAsync

```
public CompletableFuture<Void> thenRunAsync(Runnable action)
```

Returns a new CompletionStage that, when this stage completes normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

anyOf

```
static public CompletableFuture<Object> anyOf(CompletableFuture<?>... cfs)
```

Returns a new CompletableFuture that is completed when any of the given CompletableFutures complete, with the same result.

CS2030S Functional Programming Interface

```
interface Transformer<T, R> {
  R transform(T t);
}
interface BooleanCondition<T> {
  boolean test(T t);
}
interface Producer<T> {
  T produce();
}
interface Consumer<T> {
  void consume(T t);
}
interface Combiner<T,S,R> {
  R combine(T t, S s);
}
```

This page is intentionally left blank.

This page is intentionally left blank.

This page is intentionally left blank.

END OF PAPER